Yesterday 6:48 PM

ai_policy.pdf

PDF

lab_one.pdf

PDF

okay tell me detailed what to do for the step i'm listing below

Set up repo and skeleton first

Init a git repo and a Next.js app (npx create-next-app), then make your first commit before writing any feature code. This is what gets your commit history spread across days instead of arriving as one dump — a 0/18 automatic if the whole app lands in one or two commits.

Pasted text(5).txt

Document

Microsoft Windows [Version 10.0.26200.8973]

(c) Microsoft Corporation. All rights reserved.

C:\Users\hpars\OneDrive\Documents\GitHub\sdp_lab1>npm install

up to date, audited 362 packages in 4s

145 packages are looking for funding

  run npm fund for details

12 high severity vulnerabilities

To address issues that do not require attention, run:

  npm audit fix

To address all issues (including breaking changes), run:

  npm audit fix --force

Run npm audit for details.

C:\Users\hpars\OneDrive\Documents\GitHub\sdp_lab1>npm run dev

> sdp_lab1@0.1.0 dev

> next dev

▲ Next.js 16.2.12 (Turbopack)

- Local: http://localhost:3000

- Network: http://192.168.0.158:3000

✓ Ready in 3.3s

⚠ Warning: Next.js inferred your workspace root, but it may not be correct.

We detected multiple lockfiles and selected the directory of C:\Users\hpars\package-lock.json as the root directory.

To silence this warning, set turbopack.root in your Next.js config, or consider removing one of the lockfiles if it's not needed.

See https://nextjs.org/docs/app/api-reference/config/next-config-js/turbopack#root-directory for more information.

Detected additional lockfiles:

* C:\Users\hpars\OneDrive\Documents\GitHub\sdp_lab1\package-lock.json

Attention: Next.js now collects completely anonymous telemetry regarding usage.

This information is used to shape Next.js' roadmap and prioritize features.

You can learn more, including how to opt-out if you'd not like to participate in this anonymous program, by visiting the following URL:

https://nextjs.org/telemetry

Error: Turbopack build failed with 1 errors:

./OneDrive/Documents/GitHub/sdp_lab1/app

Error: Next.js inferred your workspace root, but it may not be correct.

We couldn't find the Next.js package (next/package.json) from the project directory: C:\Users\hpars\OneDrive\Documents\GitHub\sdp_lab1\app

To fix this, set turbopack.root in your Next.js config, or ensure the Next.js package is resolvable from this directory.

Note: For security and performance reasons, files outside of the project directory will not be compiled.

See https://nextjs.org/docs/app/api-reference/config/next-config-js/turbopack#root-directory for more information.

at ignore-listed frames

C:\Users\hpars\OneDrive\Documents\GitHub\sdp_lab1>npm run dev

> sdp_lab1@0.1.0 dev

> next dev

[wait] starting the development server ...

[info] waiting on http://localhost:3000 ...

Error: Configuring Next.js via 'next.config.ts' is not supported. Please replace the file with 'next.config.js'.

at Object.loadConfig [as default] (C:\Users\hpars\OneDrive\Documents\GitHub\sdp_lab1\node_modules\next\dist\next-server\server\config.js:204:19)

at new Server (C:\Users\hpars\OneDrive\Documents\GitHub\sdp_lab1\node_modules\next\dist\next-server\server\next-server.js:50:43)

at new DevServer (C:\Users\hpars\OneDrive\Documents\GitHub\sdp_lab1\node_modules\next\dist\server\next-dev-server.js:1:2851)

at createServer (C:\Users\hpars\OneDrive\Documents\GitHub\sdp_lab1\node_modules\next\dist\server\next.js:2:105)

at start (C:\Users\hpars\OneDrive\Documents\GitHub\sdp_lab1\node_modules\next\dist\server\lib\start-server.js:1:323)

at nextDev (C:\Users\hpars\OneDrive\Documents\GitHub\sdp_lab1\node_modules\next\dist\cli\next-dev.js:21:279)

at C:\Users\hpars\OneDrive\Documents\GitHub\sdp_lab1\node_modules\next\dist\bin\next:28:339

C:\Users\hpars\OneDrive\Documents\GitHub\sdp_lab1>git add .

warning: in the working copy of '.gitattributes', LF will be replaced by CRLF the next time Git touches it

warning: in the working copy of '.gitignore', LF will be replaced by CRLF the next time Git touches it

warning: in the working copy of 'AGENTS.md', LF will be replaced by CRLF the next time Git touches it

warning: in the working copy of 'CLAUDE.md', LF will be replaced by CRLF the next time Git touches it

warning: in the working copy of 'README.md', LF will be replaced by CRLF the next time Git touches it

warning: in the working copy of 'app/globals.css', LF will be replaced by CRLF the next time Git touches it

warning: in the working copy of 'app/layout.tsx', LF will be replaced by CRLF the next time Git touches it

warning: in the working copy of 'app/page.tsx', LF will be replaced by CRLF the next time Git touches it

warning: in the working copy of 'eslint.config.mjs', LF will be replaced by CRLF the next time Git touches it

warning: in the working copy of 'next.config.ts', LF will be replaced by CRLF the next time Git touches it

warning: in the working copy of 'package-lock.json', LF will be replaced by CRLF the next time Git touches it

warning: in the working copy of 'package.json', LF will be replaced by CRLF the next time Git touches it

warning: in the working copy of 'postcss.config.mjs', LF will be replaced by CRLF the next time Git touches it

warning: in the working copy of 'tsconfig.json', LF will be replaced by CRLF the next time Git touches it

C:\Users\hpars\OneDrive\Documents\GitHub\sdp_lab1>

Pasted text(6).txt

Document

import Database from "better-sqlite3";

import fs from "fs";

```
import path from "path";

console.log("Opening SQLite database...");

const dbPath = path.join(process.cwd(), "database", "database.db");
console.log("Database path:", dbPath);

const db = new Database(dbPath);

const schema = fs.readFileSync(
  path.join(process.cwd(), "database", "schema.sql"),
  "utf8"
);

db.exec(schema);

console.log("SQLite database ready.");

export default db;

{
  "compilerOptions": {
    "target": "ES2017",
    "lib": ["dom", "dom.iterable", "esnext"],
    "allowJs": true,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
```

```
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "react-jsx",
    "incremental": true,
    "plugins": [
     {
       "name": "next"
     }
    ],
    "paths": {
     "@/*": ["./*"]
    }
  },
  "include": [
    "next-env.d.ts",
    "**/*.ts",
    "**/*.tsx",
    ".next/types/**/*.ts",
    ".next/dev/types/**/*.ts",
    "**/*.mts"
  ],
  "exclude": ["node_modules"]
}
```

and there is a package-lock.json file, not sure if i must delete ot not

You're very close. The error is not about the tests—it's because Vitest doesn't understand the @ alias by default.

Let's first check something:

Do you actually have this file?

database/

schema.ts

If no, create it.

database/schema.ts

```
export const schema = `
CREATE TABLE IF NOT EXISTS tasks (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    description TEXT,
    due_date TEXT NOT NULL,
    topic TEXT NOT NULL,
    status TEXT NOT NULL DEFAULT 'todo'
        CHECK(status IN ('todo', 'in_progress', 'complete')),
    archived_at TEXT,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);
`;
```

Then fix the import

Instead of

```
import { schema } from "@/database/schema";
```

use

```
import { schema } from "../database/schema";
```

Vitest understands normal relative paths without any extra configuration.

If you DO already have database/schema.ts

Run

dir database

and paste the output.

I want to confirm you have

database

|— database.db

|— db.ts

|— schema.sql

|— schema.ts

because your error strongly suggests schema.ts doesn't exist, or it's not where the test expects it.

Once that's fixed, your tests should actually start running. If another error appears after that, paste it and we'll fix it next.

Today 3:50 PM

Step 7 — Writing the three docs alongside the code

The trap with this step is treating documentation as a wrap-up task for the last evening. Looking at the rubric again, the difference between "1 — Partial" and "2 — Complete" is entirely about accuracy and specificity against what you actually shipped — which is only possible to get right if you write each section while the thing it describes is fresh, not reconstructed from memory later.

Third-Party Code

You already did the hard thinking for this back in step 3 — this section is really just transcription plus discipline:

One entry per dependency, one line of "why" each. Not "better-sqlite3 — for the database" (too vague, reads as a bare list — that's explicitly what drops you to "1 — Partial"), but something like "better-sqlite3 — synchronous API keeps route handlers simple for a single-user local app; avoids async/await overhead an async driver would add for no benefit here."

Update it the moment you npm install something new, not at the end. It's a two-minute add each time versus a "did I use anything else?" archaeology exercise later.

Don't list things you didn't choose deliberately. If Next.js pulled in something as a transitive dependency, it doesn't need its own justified entry — this section is about your choices, not everything in package.json.

Database Design

This is where the schema decisions from step 2 get written up properly — and the rubric is explicit that it wants "tables and relationships matching the shipped schema," so the discipline here is keeping this section in sync with reality as you go, not writing an idealized version.

Describe the table(s), columns, and types — plain and factual, this part's easy since you designed it deliberately already.

Explicitly call out the three design decisions you made on purpose, because the rubric singles these out as the difference between sound and unsound: archive as a flag/timestamp rather than deletion or copying, overdue derived at read time rather than stored, and how status is constrained to the three fixed values. A sentence on why each of these is designed this way (not just that it is) is what separates "Complete" from "Partial" here.

If your schema changes mid-project — and it probably will, a little — update this section the same day, so it never drifts out of sync with what's actually in the SQLite file. A marker checking this against your shipped schema is exactly the failure mode this section exists to avoid.

Running It

This is the one most worth stress-testing yourself before submission, because the rubric's bar is unusually concrete: instructions "verified against a clean clone."

State the exact Node version you're developing against, not "a recent version of Node."

List every command in order, with nothing assumed — install, environment setup if any, run, and the test command from step 6. Someone with "nothing else to hand" per the brief's own wording needs to get from a fresh clone to a running app and a passing test suite using only what's written here.

Actually do a clean-clone dry run before submitting. Clone your own repo into a fresh folder (or ask someone else to try), follow only what's written, and see where it breaks. This is the same test the functional walkthrough opens with — "the application installs and starts by following the README alone" — so it's worth catching any gap yourself rather than losing marks on step 1 of the walkthrough.

Sequencing advice, same theme as testing

Touch all three docs in small increments across your commits rather than one big writing session — partly for accuracy, but also because a documentation-only commit or two, spread across sessions, is exactly the kind of "coherent slice... spread over more than one session" the commit-history criterion is rewarding.

Step 7 isn't coding—it's writing your documentation as you go. I'd recommend creating a docs folder now so you don't leave it until the last day.

Create this structure

docs/

|— ThirdPartyCode.md

|— DatabaseDesign.md

|— RunningIt.md

1. ThirdPartyCode.md

Put this in docs/ThirdPartyCode.md.

Third-Party Code

Next.js

Used as the web application framework. It provides routing, server actions, and React integration.

React

Used to build the user interface using reusable components.

better-sqlite3

Used to communicate with the local SQLite database. Its synchronous API keeps database code simple for a single-user local application.

Vitest

Used as the project's test runner. It provides fast unit testing and integrates well with the project's TypeScript setup.

Whenever you install another package that you chose, add one more section explaining why.

2. DatabaseDesign.md

Put this in docs/DatabaseDesign.md.

Database Design

Database

The application uses a local SQLite database stored as:

database/database.db

The project contains a single table called **tasks**.

Tasks Table

| Column | Type | Description |

|--------|------|-------------|

| id | INTEGER PRIMARY KEY AUTOINCREMENT | Unique task ID |

| title | TEXT NOT NULL | Task title |

| description | TEXT | Optional description |

| due_date | TEXT NOT NULL | Due date stored as ISO date |

| topic | TEXT NOT NULL | Subject/category |

| status | TEXT NOT NULL | Todo, In Progress or Complete |

| archived_at | TEXT | Null while active, timestamp when archived |

| created_at | TEXT | Creation timestamp |

Status Constraint

The status column is limited using a CHECK constraint.

Allowed values are:

- todo
- in_progress
- complete

This prevents invalid statuses from being stored.

Archiving

Tasks are never deleted.

Instead, the archived_at field is populated with the current timestamp.

Active tasks have:

archived_at = NULL

Archived tasks remain in the database and can still be viewed.

Overdue Tasks

Overdue is **not stored** in the database.

Instead it is calculated whenever tasks are displayed.

A task is overdue when:

- the due date has passed
- the task is not marked Complete

This avoids storing data that would become outdated over time.

3. RunningIt.md

Put this in docs/RunningIt.md.

Running the Project

Requirements

- Node.js 22.x
- npm

Install

Clone the repository.

Install dependencies:

```bash
npm install
```

Start the application

```bash
npm run dev
```

Open:

http://localhost:3000

Run the tests

```bash
npm test
```

Database

The SQLite database is stored locally at:

database/database.db

If the file does not exist, it is created automatically when the application first starts.

Before submission

Once your project is finished, do one final check:

Delete node_modules.

Delete database/database.db (optional if your app recreates it).

Run:

npm install

npm run dev

npm test

If everything works from those commands, your "Running It" documentation is accurate.

After Step 7, the only major remaining work should be polishing the UI, completing any remaining assignment features, and making sure your commit history and documentation are in sync with the final code.

For clarity of the submission, please have your documentation in the repo under a folder called 'docs' or something similar. Please submit the transcripts the same way under an 'ai' folder.

That means your submission should be just the repo link. - from the lecturer

Step 8 — AI usage tracking and attribution

Short guide

Capture as you go, not after. For chat-based tools, that means exporting or saving the conversation itself (like this one) rather than trying to reconstruct "what did I ask the AI" from memory later — the policy expects transcripts, not a paraphrase.

Know which of the three "AI context" buckets each session falls into, because they're documented differently:

Writing (planning, docs, brainstorming) → needs a footnote/declaration in the document itself, not per-chunk attribution.

Code generation → needs a Assisted-by: Tool[Model] line on the specific commit where you used it.

In-line/autocomplete or code review → doesn't need per-commit tagging, but does need a one-time declaration in the README.

Every document and every repo needs a declaration either way — even "not used," per the policy's non-usage wording. A document with no declaration at all isn't marked.

Record what you used it for, not just that you used it — tool, model, purpose (planning / code-gen / review / editing) is the minimum triplet the policy asks for.

When a chat corrects a wrong or unsuitable AI output, keep that in the transcript rather than editing it out. That moment of catching-and-redirecting is specifically what separates full marks from partial on the AI-usage criterion — a transcript that's all agreement with no correction reads as "no evidence of the author redirecting an output."

Start a fresh session per distinct piece of work where practical, especially for anything assessment-related — it keeps each transcript legible as "this is what happened for this task" rather than one sprawling log covering the whole project.

What to tell a different chat, for future projects

Open a new chat with something like:

"I'm working under a university AI policy that requires me to declare AI usage with: tool name, model name, and what I used it for (planning / code generation / editing / review). For code, I need Assisted-by: Tool[Model] on any commit where AI-generated code was used. I want you to help me think things through and catch me if you disagree with an approach — I'll be keeping this transcript for submission, so be direct rather than just agreeing with me."

That last line matters more than it looks — it primes the chat to actually push back, which is what generates the "author redirecting an unsuitable output" moments the rubric wants evidence of, rather than a transcript of pure agreement.

Your lecturer's note changes how you should organize the repo. Here's what I'd do.

Folder structure

SDP-Lab1/

|

├── app/

├── database/

├── lib/

├── tests/

├── docs/

| ├── ThirdPartyCode.md

| ├── DatabaseDesign.md

| └── RunningIt.md

|

├── ai/

| ├── planning-chat.pdf

| ├── database-chat.pdf

| ├── implementation-chat.pdf

| ├── testing-chat.pdf

| └── README.md

|

├── README.md

└── package.json

Step 8

This isn't writing code—it's collecting evidence of how you used AI.

1. Save your ChatGPT conversations

Since you've been using ChatGPT throughout this project, export or save the relevant chats.

You don't have to invent anything afterwards.

For example:

ai/

|— 01-project-planning.pdf

|— 02-database-design.pdf

|— 03-create-edit-archive.pdf

|— 04-sorting-overdue.pdf

|— 05-testing.pdf

If one long conversation covers everything (like this one), that's also perfectly reasonable unless your lecturer specifically asked for separate transcripts.

2. Create ai/README.md

Something like this is enough:

AI Usage

Tool used:

- ChatGPT

Model:

- GPT-5.5

Purpose:

- Project planning

- Database schema design

- Code generation

- Code review

- Debugging

- Documentation assistance

The attached transcript(s) contain the complete interaction history used during development.

3. Add AI declarations to documentation

At the bottom of each document in docs/, add something like:

DatabaseDesign.md

AI Declaration

This document was drafted with assistance from ChatGPT (GPT-5.5). The author reviewed and edited the final content.

Do the same for:

ThirdPartyCode.md

RunningIt.md

4. README declaration

Since your lecturer referenced the university AI policy, add a section to your root README.md:

AI Usage Declaration

ChatGPT (GPT-5.5) was used during this project for:

- Planning

- Explaining concepts

- Debugging

- Reviewing code

- Generating draft code

- Assisting with documentation

All generated code was reviewed, tested and modified by the author before submission.

5. Git commit messages

The policy says code-generation commits should be marked.

For commits where ChatGPT helped generate code, use commit messages like:

Implement SQLite persistence

Assisted-by: ChatGPT [GPT-5.5]

or

Add archive functionality

Assisted-by: ChatGPT [GPT-5.5]

If those commits have already been pushed, don't rewrite history just to add the footer unless your lecturer specifically requires it. From this point onward, include it on commits where AI-generated code is a significant part of the change.

Before submitting

Your final repository should look roughly like this:

SDP-Lab1/

|

├── app/

```
├── database/
├── lib/
├── tests/
├── docs/
|   ├── ThirdPartyCode.md
|   ├── DatabaseDesign.md
|   └── RunningIt.md
|
├── ai/
|   ├── README.md
|   └── chat-transcript(s).pdf
|
├── README.md
├── package.json
└── ...
```

That matches your lecturer's request to keep documentation under docs/ and AI transcripts under ai/, so your submission can simply be the repository link.